45 TYPEORM VS WRITING SQL — WHEN YOU DON'T WRITE SELECT *, AND WHEN YOU DO

DEFINITION:

> TypeORM uses TypeScript/JavaScript methods and entities to interact with databases, generating SQL queries automatically for most CRUD operations.

In TypeORM you normally do NOT write SELECT * / INSERT INTO / UPDATE / DELETE yourself. You call TypeScript methods on a Repository and TypeORM generates the SQL. But TypeORM reduces SQL, it doesn't remove it: complex queries still use QueryBuilder or raw SQL.

theory/43 shows how the SQL is generated in detail. This file is the quick practical guide: what replaces each SQL statement, and which tool to use as queries get more complex.

Full stack this fits into:

Angular	→	NestJS	→	TypeORM	→	Supabase PostgreSQL
(UI)		(API)		(writes SQL)		(runs SQL)

SECTION 1 — SQL STATEMENT → TYPEORM METHOD

Using a `products` table (Product entity) as the example:

SQL you would write	TypeORM call
SELECT * FROM products	productRepo.find()
SELECT * FROM products WHERE id = 1	productRepo.findOneBy({ id: 1 })
SELECT * FROM products WHERE category = 'x'	productRepo.findBy({ category: 'x' })
SELECT COUNT(*) FROM products	productRepo.count()
INSERT INTO products (...) VALUES (...)	productRepo.save(product)
UPDATE products SET price = 500 WHERE id = 1	productRepo.update(1, { price: 500 })
DELETE FROM products WHERE id = 1	productRepo.delete(1)

```
Same thing in THIS project (student-pg.service.ts):
  studentRepo.find()              SELECT ... FROM students
  studentRepo.findOneBy({ id })   SELECT ... FROM students WHERE id = $1 LIMIT 1
  studentRepo.save(student)       INSERT ... RETURNING ...   /  UPDATE ...
  studentRepo.delete(id)          DELETE FROM students WHERE id = $1
```

Small detail: TypeORM never literally sends `SELECT *`. It lists every entity column by name: SELECT "id", "name", "age", "email", "course", "createdAt", "updatedAt" FROM "students" Same result, but it only reads columns the entity knows about.

SECTION 2 — WHAT HAPPENS BEHIND ONE CALL

```
  const products = await this.productRepo.find();

  1. TypeORM looks at the Product entity -> table "products", its columns
  2. builds SQL:  SELECT "id", "name", "price" ... FROM "products"
  3. pg driver sends it to PostgreSQL on Supabase
  4. Postgres returns rows
  5. TypeORM turns each row into a Product object
  6. you get Product[] — typed, autocompleted, no SQL strings in your code

save() vs update():
  save(entity)          loads/compares first, then INSERT or UPDATE;
                        runs entity hooks + @UpdateDateColumn
  update(id, fields)    one direct UPDATE, no loading, faster, but
                        skips entity hooks and returns no entity
```

SECTION 3 — WHEN IT GETS COMPLEX: TWO OPTIONS

OPTION 1 — QUERYBUILDER (still TypeScript, still safe)

```
"Top 10 products costing more than 1000, name contains 'laptop',
 most expensive first":

  const products = await this.productRepo
    .createQueryBuilder('p')                                  // 'p' = alias for products
    .where('p.price > :price', { price: 1000 })
    .andWhere('p.name ILIKE :name', { name: '%laptop%' })     // ILIKE = case-insensitive (Postgres)
    .orderBy('p.price', 'DESC')
    .take(10)
    .getMany();
```

SQL TypeORM generates:

```
SELECT ... FROM "products" "p"
WHERE "p"."price" > $1 AND "p"."name" ILIKE $2
ORDER BY "p"."price" DESC
LIMIT 10                          -- params: [1000, '%laptop%']
```

Use for: joins, dynamic filters (add .andWhere only if a filter was sent), GROUP BY, counts per group, pagination with joins.

Same example with students:

```
this.studentRepo
  .createQueryBuilder('s')
  .where('s.age > :age', { age: 20 })
  .andWhere('s.course = :course', { course: 'Computer Science' })
```

```
  .orderBy('s.name', 'ASC')
  .getMany();
```

OPTION 2 — RAW SQL (you write the SQL string)

For reports with CTEs, window functions, or Postgres-only features:

```
const result = await this.dataSource.query(`
  SELECT name, SUM(price) AS total
  FROM products
  GROUP BY name
  ORDER BY total DESC
`);

// with user input -> ALWAYS $1 parameters
await this.dataSource.query(
  'SELECT * FROM products WHERE price > $1',
  [minPrice],
);
```

To use dataSource in a service:

```
constructor(private dataSource: DataSource) {}   // import { DataSource } from 'typeorm'
```

Raw SQL returns plain objects (no entity mapping) — fine for reports.

SECTION 4 — WHICH ONE TO USE

Query type	Recommended
Basic CRUD	Repository (find, save, update, delete)
Filters, sorting, pagination	Repository options or QueryBuilder
Joins and aggregations	QueryBuilder or raw SQL
Very complex analytics / reports	Often raw SQL

Repository options already cover a lot before you need QueryBuilder:

```
  this.productRepo.find({
    where: { price: MoreThan(1000), name: ILike('%laptop%') },
    order: { price: 'DESC' },
    take: 10,
  });
(same query as Section 3's QueryBuilder example)
```

SECTION 5 — PROS AND CONS OF LETTING TYPEORM WRITE SQL

- less code, no SQL strings for everyday CRUD
- type-safe: rename a property and the compiler finds every use
- parameters ($1) added automatically → safe from SQL injection
- same code works on Postgres, MySQL, SQLite (mostly)

- generated SQL can be less efficient for complex queries
- easy to forget what SQL actually runs → turn on logging: true
- very advanced Postgres features still need raw SQL

★ BOTTOM LINE

TypeORM writes the SQL for everyday work: Repository methods for CRUD, QueryBuilder for complex-but-dynamic queries, raw SQL for heavy reports. It reduces the SQL you write, it doesn't eliminate it — so use TypeORM for CRUD, and still learn SQL (**theory/37**) for complex reports and for understanding what TypeORM is doing for you.

See also: **theory/43-where-sql-queries-come-from.txt** (full detail on generated SQL, logging, transactions), **theory/44-how-the-table-was-created.txt** (CREATE TABLE / synchronize), **theory/37-sql-cheatsheet.txt** (SQL itself), **theory/42-ways-to-connect-postgres.txt** (other libraries)